CS456 – Modern Cybersecurity

Mini Group Project 1: Resilient Security Systems – Part 2 (Recovery)

Objective

In Part 1, your team established the foundation of system availability: running a security-critical service, modeling unpredictable process termination, and deploying an independent heartbeat monitor to detect failures in real time. However, detection without automated recovery leaves a service unavailable. If an authentication server, firewall agent, or key-storage daemon crashes, an alert informs administrators of downtime, but authorized clients remain locked out until human intervention occurs.

In Part 2, your team will complete the availability lifecycle by implementing **Architectural Fault Recovery Tactics**: introducing process replication, automated failover, and state preservation to guarantee zero or near-zero downtime.

Background

Once a fault is detected via the heartbeat mechanism, the supervisory architecture must execute recovery actions to restore operational capability. High-availability distributed systems rely on two primary redundancy models:

1. Passive Redundancy (Warm/Cold Standby with Checkpointing)

- **Architecture:** A **Primary Node** handles all live client traffic while a **Standby Backup Node** remains idle or dormant.
- **State Preservation (Checkpointing):** The primary node periodically snapshots its operational state (e.g., active session tokens, blocklists, issued credentials) to shared storage or syncs incremental state snapshots directly to the standby node.
- **Failover Mechanism:** When the monitor detects that the primary node has died (via missed heartbeats), it triggers a failover sequence:
    1. Signals or spawns the standby node.
    2. The standby node loads the most recent checkpoint.
    3. The standby assumes the primary identity/address and resumes serving incoming client traffic.

- **Recovery Trade-off:** Minimal idle computational overhead, but incurs a small non-zero failover delay (Mean Time to Repair/MTTR) and potential loss of state updates that occurred after the last checkpoint.

2. Active Redundancy (Hot Spare/Replicated Execution)

- **Architecture:** Both **Primary** and **Secondary** nodes run concurrently in active memory.
- **State Synchronization:** Incoming client requests are replicated or broadcast so that both nodes process operations or update their internal state in parallel.
- **Failover Mechanism:** Because the secondary node maintains lock-step state, the moment the monitor flags a primary crash, client traffic immediately routes to the secondary with instantaneous (zero-delay) failover.
- **Recovery Trade-off:** Instantaneous recovery and zero lost transactions, but requires double the active system resources (CPU/memory) and introduces network overhead to keep states synchronized.

3. Cybersecurity Considerations in Automated Recovery

Automated recovery scripts introduce serious security risks if poorly architected:

- **Split-Brain Syndrome:** If network latency causes the monitor to prematurely declare the primary dead while it is still running, both nodes may believe they are the authoritative primary, causing conflicting state updates or corrupted security policies.
- **Poison-Pill Payloads:** If the primary crashes due to a malicious malformed client request, naive failover may feed that exact same packet to the secondary, immediately crashing it as well (cascading failure).
- **State Integrity:** Checkpoint files must be protected against local tampering; an attacker with filesystem access could inject elevated privileges directly into a checkpoint state before failover occurs.

Project Instructions

Your team will extend your Part 1 codebase into a fault-tolerant cluster.

Task 1: Select your Redundancy Model

Choose and implement **one** of the two redundancy models:

- **Option A: Passive Redundancy (Warm Standby + Checkpointing):** Primary periodically exports state to a local file or socket. Upon primary crash detection, the monitor activates the backup, which loads the latest checkpoint and takes over client traffic.

- **Option B: Active Redundancy (Hot Spare + State Sync):** Both nodes run simultaneously. Client requests update state across both nodes. Upon primary crash detection, the monitor promotes the secondary node to handle client traffic with zero downtime.

Task 2: Implement Dynamic State Handling

Your security service must maintain dynamic internal state that changes across requests:

- Demonstrate that the state accumulated on the primary node **survives the crash** and is present on the recovered node.
- Clients querying the service *after* the crash must receive accurate responses reflecting pre-crash operations.

Task 3: Automated Failover Coordination

Decouple failover from manual human action: **the Monitor must orchestrate or trigger the switch automatically**.

When the primary crashes non-deterministically:

1. The monitor flags the threshold timeout (from Part 1).
2. The monitor issues a promotion signal or directly launches the failover process.
3. The failover logs clear timestamps showing: [Crash Detected] -> [Failover Triggered] -> [Service Restored].

Task 4: Adversarial Assessment

The red-team auditor on the team will evaluate failover resilience:

- **Attack Scenario 1 (Induced False Failover):** The auditor simulates network delay or drops heartbeat packets while the primary is still alive. Does the system accidentally spin up two concurrent conflicting primaries (split-brain)?
- **Attack Scenario 2 (Tampered Checkpoint/Desync):** For passive systems, what happens if the checkpoint file is corrupted? For active systems, what happens if the sync channel between nodes is blocked?

Document findings and evaluate whether the system handled the disruption gracefully.

Reflection (individual)

Take a few minutes to reflect on your personal workflow, the concepts you encountered, and the tools you used to complete this assignment.

1. How difficult was this assignment on a scale of 1 (very easy) – 5 (very challenging)?
2. What was easy about this assignment and why?
3. What was challenging about this assignment and why?

4. What external tools or resources did you use to complete this assignment and where were they helpful?

Deliverables

Please submit the following via Canvas:

1. Each team must submit a single repository link (e.g., GitHub/GitLab) containing the original Part 1 scripts and the following:
    a. **Replica/Secondary Process.**
    b. **State Sync/Checkpointing module.**
    c. **Updated Monitor Failover Controller.**
    d. **Adversarial Audit script.**
2. Each team must submit a single PDF report containing:
    a. **Names of all teammates:** First and last.
    b. **Software Architectural Diagram:** A clear graphical diagram illustrating the interaction between Client, Primary Node, Secondary Node, Monitor, and Checkpoint/Sync channels.
    c. **Redundancy Justification:** Explain why your team selected Active vs. Passive redundancy, detailing the trade-offs between synchronization overhead, implementation complexity, and Mean Time to Repair (MTTR).
    d. **Execution & Setup Instructions:** Unambiguous, step-by-step terminal commands to compile, configure, and execute the service, monitor, and attack script.
    e. **Adversarial Audit Summary:** Report on the auditor's attempt to trigger a split-brain condition or corrupt failover synchronization.
    f. **Contributions:** File CONTRIBUTIONS.md detailing each member's role for Part 2.
3. Each team must submit a single video demonstration that meets the following criteria:
    a. Time Limit: Strictly under 3 minutes (anything over will not be reviewed). If Canvas doesn't allow the video upload, please provide a Google Drive or unlisted YouTube link.
    b. Show the Client, Primary, and Monitor running and non-deterministically failing.
    c. Show the monitor detecting the missed heartbeats, triggering automated failover, and promoting the secondary node.
    d. Demonstrate that pre-crash state was retained and operations continue without manual intervention.
    e. Voiceover: Explain what is going on.
4. Each individual (separately in Canvas) must submit their reflection with the four reflection questions.

Note: For team deliverables, one person may submit. For individual reflections, each individual must submit.

Grading

Here is the grading scheme for the project report:

- Redundancy implementation script(s) (20)
- Adversarial script (20)
- PDF report (20)
- Video demo (20)
- Individual reflection (20)

TOTAL: 100 points